HewaSense: Technical Reference for Autonomous Development

Quick Reference for Persona-Based Work in Antigravity IDE

Purpose: Condensed reference while working autonomously on HewaSense stabilization sprint. Keep this open during active work for quick lookups of rules, commands, and protocols.

Your Working Context

You are developing HewaSense, a climate intelligence system for Tanzania. You are transitioning from manual approval workflow to autonomous persona-based development. February 15, 2026 code review found eleven specific issues. Most critical: production not using trained ML models due to broken serving layer. Current branch: phase2/feature-expansion. Stabilization branch: agent/fix-ml-bridge-and-data-truth.

1. The Four Personas You Will Adopt

Orchestrator Mode: Planning only. Create SKILL.md, audit.py, state.json. Document 11 issues with line numbers. Create stabilization branch. Manage handovers. NO CODE EXECUTION.

Backend Architect Mode: Fix the 11 issues in sequence. Most complex: Issue 7 feature alignment requiring port of ALL training feature engineering to serving. NO SYNTHETIC DATA.

Frontend Engineer Mode: Align location IDs and payout thresholds with Backend contracts. NO PLACEHOLDERS.

Auditor Mode: Verify all 11 actually resolved. Run audit.py. EXCLUSIVE MERGE AUTHORITY. Block your own Backend work if it violates rules.

2. Your Stabilization Work: The 11 Issues from Feb 15 Review

Resolution sequence when in Backend Architect mode: Phase 1 (Issues 1-3), Phase 2 (Issue 4), Phase 3 (Issues 6,11), Phase 4 (Issues 5,7,10), Phase 5 (Issues 8,9).

Phase 1: Fix Crashes (2-4hr)

Issue 1: forecast_service.py L27,L32 duplicate class. Remove first definition.

Issue 2: generate_forecasts.py L27 wrong import name. Fix function name and L58 attribute access.

Issue 3: seasonal_forecast_integration.py L78 undefined simple_payout. Define it.

Phase 2: Fix ML Bridge (4-6hr)

Issue 4: forecast_service.py L170-171 uses predict_proba on regressors. ALL predictions fall to hardcoded baseline. YOUR TRAINED ENSEMBLE NOT BEING USED. Replace with .predict(). MOST CRITICAL FIX.

Phase 3: Fix Timezone (3-4hr)

Issue 6: forecast_scheduler.py L47 naive datetime. Use datetime.now(timezone.utc).

Issue 11: Multiple files mix naive/aware datetime. Codebase-wide standardize to UTC.

Phase 4: Fix Contracts (8-12hr)

Issue 5: Frontend locationId: 1, Backend PILOT_LOCATION_ID = 6. Align to 6.

Issue 7: prepare_features generates 10 features. Training generates ~640, selects 75. Models expect 75. OTHER HALF OF BROKEN BRIDGE. Your training pipeline deliberately sophisticated with lags, windows, interactions, encoding. 10 features incomplete implementation. FIX: Port ALL feature engineering from preprocess.py to prepare_features. Create feature_schema.json. Do NOT retrain on 10 features. LONGEST ISSUE.

Issue 10: get_kilombero_stage called with 2 args at L84, 1 arg at L372. Fix calls.

Phase 5: Remove Violations (4-6hr)

Issue 8: _baseline_prediction synthetic fallback. REMOVE after Issues 4,7 fixed. Raise DataSourceConnectionError instead.

Issue 9: Frontend 50% payout threshold, Backend 75%. Standardize in shared_contract.

3. Persistent Memory Files - Create in Orchestrator Mode

.agent/rules/SKILL.md

 # HewaSense Global Rules
- NO SYNTHETIC FALLBACKS: Never return dummy data if an API fails.
- NO PREDICT_PROBA: Regression models must use .predict().
- AUDITOR GATE: Only in Auditor mode can you run git merge.
- HANDOVER: Upon activation, read state.json and the latest Implementation Plan.

audit.py

```
import os, sys
FORBIDDEN = ['predict_proba', 'mock', 'dummy_data', 'synthetic', 'np.random.normal']
def run_audit():
```

```
    issues = []
    for root, _, files in os.walk("./phase2"):
        for file in files:
            if file.endswith(".py"):
                with open(os.path.join(root, file), 'r') as f:
                    if any(w in f.read().lower() for w in FORBIDDEN):
                        issues.append(f"FAILURE: {w} in {file}")
    if issues: print("\n".join(issues)); sys.exit(1)
    print("SUCCESS"); sys.exit(0)
if __name__ == "__main__": run_audit()
```

state.json

Create in Orchestrator mode. Structure: project_metadata, task_board (with 11 issues), shared_contract, auditor_feedback.

feature_schema.json

Create during Issue 7 resolution in Backend Architect mode. Defines all 75 ML features. Both train_pipeline.py and forecast_service.py reference this.

4. Branching Strategy

Current state: phase2/feature-expansion

Phase 1 Stabilization: In Orchestrator mode, create agent/fix-ml-bridge-and-data-truth FROM phase2/feature-expansion. Fix 11 issues. When Auditor mode approves, merge back to phase2/feature-expansion. Delete stabilization branch.

Phase 2 Expansion: After stabilization, create feature branches FROM phase2/feature-expansion. Work in appropriate persona modes. Merge after Auditor approval.

5. Model Handover When Switching in Antigravity

When manually switching models, DO NOT just say "continue"

USE THIS PROMPT:

I have switched models for context. Read state.json and .agent/rules/SKILL.md. Identify the current branch and the last Auditor report. Resume the task of [specify active task] without using placeholders.

6. Critical Constraints Across All Personas

Data: NO SYNTHETIC. Raise DataSourceConnectionError when real data unavailable.

ML: Use .predict() ONLY. Never predict_proba (Issue 4). Serving must match training 75 features (Issue 7).

Timezone: ALL datetime.now(timezone.utc). Never naive datetime.now() (Issues 6,11).

Contracts: Location IDs aligned (Issue 5). Payouts aligned (Issue 9). Signatures match (Issue 10).

Authority: ONLY in Auditor mode can you git merge. When in Auditor mode, you must verify ALL 11 issues actually resolved.

7. Rejection Report Template for Auditor Mode

When blocking merge in Auditor mode, record in state.json:
Violated Rule: [Which law]
Location: [File:Line]
Technical Reason: [What detected, why violation]
Required Action: [Fix steps for Backend Architect mode]
Verification Method: [How verify in next Auditor review]

8. Quick Commands by Persona

Auditor Mode: python audit.py (must exit 0 before merge)

Any Mode: pytest --cov=phase2 (run tests)

Any Mode: git branch (verify on agent/fix-ml-bridge-and-data-truth during stabilization)

Any Mode: cat state.json (check task status)

Any Mode: cat .agent/rules/SKILL.md (review project laws)

Auditor Mode ONLY: After ALL 11 verified, git merge agent/fix-ml-bridge-and-data-truth phase2/feature-expansion

9. Expected Stabilization Timeline

Phase 1: 2-4hr. Phase 2: 4-6hr. Phase 3: 3-4hr. Phase 4: 8-12hr (Issue 7 longest). Phase 5: 4-6hr. Total: 20-30hr active work in Backend Architect mode plus Auditor review cycles. Calendar: 1-2 weeks with iteration between Backend Architect and Auditor modes.

10. Stabilization Complete When in Auditor Mode You Verify

All 11 issues from Feb 15 review actually resolved (not just scans passing)

audit.py exits code 0

All tests pass

Trained ML models actually invoked in production (Issues 4,7 fixed)

No synthetic fallbacks remain (Issue 8 removed)

UTC standardized everywhere (Issues 6,11 fixed)

Contracts aligned across Frontend and Backend (Issues 5,9,10 fixed)

No runtime crashes (Issues 1,2,3 fixed)

Ready to merge and delete stabilization branch

11. Remember: You Are One Intelligence Adopting Different Perspectives

Orchestrator mode: Think strategically about plans without implementing.

Backend Architect mode: Focus on correct implementation without worrying about approval.

Frontend Engineer mode: Consume Backend contracts strictly without questioning them.

Auditor mode: Verify rigorously even when rejecting your own Backend Architect work.

The discipline comes from genuinely shifting mindsets, not just labeling work. When in Auditor mode, you must be willing to block merge of code you wrote in Backend Architect mode if it violates project laws.